

Análise de Documentos com Auxílio de Agentes de IA

1. Visão Geral e Objetivo Principal

Desenvolver uma plataforma de software em Python, denominada inicialmente “IA Assistente”, para servir como hub de Agentes de IA especializados e Chats inteligentes. O objetivo é otimizar rotinas de análise e processos investigativos da Polícia Federal, começando com um módulo de Análise de Documentos.

O módulo inaugural será focado em automatizar a extração, estruturação e sumarização de informações de Notícias-Crime e outros documentos recepcionados pela Corregedoria (COR/SR/PF/SP). A plataforma deve priorizar a produção de informações factuais e neutras, garantir a segurança e a privacidade de dados sensíveis e fornecer uma interface de usuário robusta, segura e eficiente.

2. Escopo do Projeto

2.1 O que SERÁ entregue

1. Módulo Avançado de Processamento de PDF:
 - Pipeline para extração de texto de múltiplos arquivos PDF.
 - Classificação de páginas por relevância (TF-IDF) e similaridade textual (embeddings).
 - Agrupamento de conteúdo relevante, com otimização e controle de limite de tokens para análise por LLM.
2. Módulo de Análise de Conteúdo com IA:
 - Integração inicial com a API da OpenAI como provedor LLM para análise estruturada de texto.
 - A arquitetura será projetada para permitir a futura inclusão de outros provedores.
 - Sistema de gerenciamento de prompts dinâmicos carregados de um backend (Firestore).
 - Estruturação da resposta da IA em formato JSON, seguindo um modelo Pydantic pré-definido.
 - Capacidade de anonimização de dados sensíveis (NER) antes do envio para a API externa.
3. Módulo de Geração de Documentos via Templates:
 - Funcionalidade de exportação dos dados analisados para arquivos .docx.
 - Suporte para exportação baseada em templates .docx customizáveis pelo usuário, com preenchimento automático de placeholders.
4. Interface de Usuário Robusta e Segura (Flet):
 - Interface desktop/web completa com autenticação de usuário (login, signup, verificação de e-mail, reset de senha).
 - Gerenciamento de perfil de usuário e configurações (Proxy, Provedores de LLM).
 - Fluxo de trabalho guiado para upload, processamento, análise e exportação de documentos.

- Sistema de feedback para o usuário validar, editar e avaliar a precisão das análises da IA.
5. Infraestrutura de Backend e Segurança:
- Backend na nuvem (Firebase) para autenticação, armazenamento seguro de chaves de API, prompts e métricas.
 - Armazenamento seguro de credenciais locais (chaves de serviço e de criptografia) utilizando Keyring e criptografia Fernet.
 - Sistema de logging robusto com saídas locais e em nuvem (específico por usuário).

2.2 O que NÃO será entregue (inicialmente)

As seguintes funcionalidades estão fora do escopo das fases iniciais (1 e 2), mas constam no roadmap futuro:

- * Sistema de OCR integrado para processar documentos baseados em imagem.
- * Integração direta com sistemas de gestão processual externos (ex: ePol).
- * Suporte completo a LLMs executados localmente (off-line).
- * Interface para integração com ferramentas de RPA.

3. Tecnologias Utilizadas

Core / Backend:

- Python 3.12+
- LangChain / langchain-openai (Orquestração de IA)
- OpenAI API (Provedor LLM inicial da Fase 2)
- PyMuPDF (fitz), pdfplumber (Processamento de PDF)
- Sentence-Transformers, scikit-learn (Embeddings e TF-IDF)
- Tiktoken (Tokenização)

Interface:

- Flet (Framework para UI em Python)

Cloud / Autenticação:

- Firebase (Authentication, Firestore, Cloud Storage)

Segurança:

- cryptography (Criptografia de chaves Fernet)
- keyring (Armazenamento seguro de credenciais no SO)

4. Fases do Projeto (Roadmap)

O projeto será desenvolvido em fases incrementais para permitir entregas de valor contínuas e validação com os usuários.

➤ **FASE 1:** Prova de Conceito - MVP Básico

Objetivo: Validar a viabilidade técnica do fluxo principal.

Entregas:

- Extração de texto de arquivo PDF.
- Análise via LLM com prompt único fixo.
- Interface de usuário mínima para upload e exibição de resultado em texto.
- Exportação simples para arquivo .docx.

➤ **FASE 2:** MVP Completo e Funcional (Entrega Alvo Inicial)

Objetivo: Entregar um Módulo-Agente de IA especializado, funcional e distribuível, com interface de usuário completa e infraestrutura de backend robusta.

Entregas:

- Análise de PDF Avançada: Pipeline completo de extração, classificação, agrupamento e agregação de texto de múltiplos arquivos.
- Análise com LLM: Integração exclusiva com a API da OpenAI como provedor de LLM para esta fase, realizando a análise estruturada de documentos com base em prompts segmentados e dinâmicos.
- Interface Flet Completa: GUI com autenticação de usuário (login, signup, reset), gerenciamento de perfil, configurações de proxy e LLM, e um fluxo de trabalho guiado.
- Gerenciamento Seguro de Chaves: Armazenamento seguro de chaves de serviço e chaves de API do usuário usando Keyring e criptografia.
- Exportação Avançada: Funcionalidade de exportação para DOCX, tanto simples quanto baseada em templates customizáveis.
- Logging e Métricas: Sistema robusto de logging local/nuvem e coleta de métricas de uso e feedback.

➤ **FASE 3:** RAG e Interação Avançada

Objetivo: Implementar Retrieval-Augmented Generation (RAG) para melhorar a contextualização e permitir interação conversacional com documentos.

Entregas:

- Implementação de RAG "on-the-fly" para indexação vetorial de documentos em memória. RAG tradicional processado localmente ou via serviço da Openai.
- Implementação de RAG avançado otimizado por sumarização de conteúdo (graphs)
- Desenvolvimento da interface "Chat com PDF".

➤ **FASE 4:** Segurança e Atualizações

Objetivo: Incrementar a segurança dos dados e facilitar a distribuição de novas versões.

Entregas:

- Implementação de sistema de anonimização de dados sensíveis (NER).

- Desenvolvimento de um atualizador automático (Updater.exe).
- **FASE 5: Integração Externa (RPA)**

Objetivo: Permitir que processos de RPA interajam com as funcionalidades da aplicação.

Entregas:

 - Desenvolvimento de uma interface (API HTTP local ou Firebase Functions) para integração.
- **FASE 6: Processamento Offline (LLMs Locais)**

Objetivo: Oferecer a opção de processamento totalmente offline.

Entregas:

 - Integração para execução de LLMs locais (via Llama.cpp, Ollama, etc).
 - Interface para gerenciamento de modelos locais.

5. Critérios de Aceitação

- A aplicação deve ser empacotável e executável em ambiente Windows 10/11.
- O sistema de autenticação deve validar o domínio de e-mail e o status de verificação.
- O pipeline de processamento de PDF deve extrair texto de forma consistente para documentos de diferentes layouts.
- As chaves de API e de serviço devem ser armazenadas de forma criptografada, não em texto plano.
- A interface deve permitir o ciclo completo: carregar arquivo(s) -> processar -> analisar -> editar resultado -> exportar.
- O sistema de feedback deve registrar as edições do usuário para cálculo de métricas de precisão da IA.

6. Métricas de Resultado

- Taxa de detecção correta de páginas relevantes: > 90%
- Tempo médio de processamento e análise por documento: Redução significativa em comparação com o processo manual.
- Índice de acerto da IA: Medido pela similaridade entre a resposta original da LLM e a versão final editada pelo analista.
- Taxa de adoção: Número de usuários ativos e frequência de uso.
- Economia de tempo/mão-de-obra: Comparativo quantitativo do tempo gasto na análise manual vs. com a ferramenta.

7. Riscos Identificados

1. Variação na Qualidade dos Documentos de Entrada:

- Descrição: PDFs de baixa qualidade, escaneados ou com layouts complexos podem dificultar a extração de texto.
- Mitigação: Implementar um sistema robusto de pré-processamento, detecção de qualidade e, futuramente, integração com OCR.

2. Limites de Tokens e Custo das APIs de IA:

- Descrição: Documentos muito longos podem exceder o limite de contexto dos modelos, e o uso contínuo gera custos.
- Mitigação: Implementar *chunking* inteligente, otimização de tokens, sistema de cache para prompts e monitoramento de custos.

3. Privacidade e Segurança dos Dados:

- Descrição: Envio de dados sensíveis para APIs de terceiros.
- Mitigação: Priorizar o desenvolvimento do módulo de anonimização (NER). Validar as políticas de privacidade dos provedores de IA. Garantir criptografia de ponta a ponta para credenciais.

4. Complexidade da Compilação e Distribuição:

- Descrição: Dependências pesadas (PyTorch, Transformers) tornam o processo de compilação com PyInstaller lento e propenso a erros.
- Mitigação: Criar pipelines de build e debug separados. Documentar soluções para problemas comuns de empacotamento (.spec file, assets, etc.). Desenvolver um atualizador automático.

5. Gerenciamento de Segredos no Ambiente Cliente:

- Descrição: A aplicação precisa armazenar e acessar chaves de API e de criptografia de forma segura na máquina do usuário.
- Mitigação: Utilizar uma abordagem híbrida com keyring (para a chave de criptografia principal) e arquivos locais criptografados com Fernet (para segredos maiores como a chave de serviço).